FreeFlow v1.0.1-rc 结构化导入与复制粘贴架构改造清单

1. 文档目标

本文件用于定义一套新的“结构化导入与复制粘贴架构”改造范围。

本次文档覆盖内容包括：

- 现有版本复制 / 粘贴 / 拖拽导入链路的实际情况
- 新架构的完整目标形态
- 与当前实现的逐项差异
- 可改造模块清单
- 分阶段实施建议
- 兼容策略、风险点与收益评估

本文件明确约束：

- 当前版本先作为兼容层保留，不直接推翻
- 本次设计先不加入 AI 后处理、AI 自动整理、AI 语义增强
- 本次重点只做“外部内容导入、内部复制粘贴、统一结构协议、原生渲染落地”

---

2. 改造总目标

当前 FreeFlow 已具备基本的内部复制、外部 HTML / 文本粘贴、文件拖拽导入能力，但整体仍属于“入口分散、结构不统一、语义保留有限”的实现。

本次架构改造的目标，不是继续在旧链路上堆特判，而是把复制 / 粘贴 / 拖拽导入统一升级为一套可扩展的结构化系统。

目标状态如下：

- 所有外部来源先进入统一输入网关
- 不同来源由各自解析器负责清洗、提取、保留结构
- 所有解析结果统一落为 FreeFlow 内部唯一标准 JSON 协议
- 所有渲染都从标准 JSON 出发，而不是直接吃原始 HTML
- 渲染结果全部变成 FreeFlow 原生可编辑元素
- 内部复制、外部粘贴、右键粘贴、拖拽导入逐步收口到同一主链

---

3. 本次设计范围

3.1 本次纳入改造范围

- 外部内容输入网关
- 来源识别与导入调度
- HTML / 纯文本 / 网页正文 / Office 富文本 / 代码文本解析
- 统一中间 JSON 协议
- 原生文本块 / 段落 / 列表 / 表格 / 代码块 / 图片元素渲染
- 内部复制协议与外部导入协议的边界统一
- Ctrl + V、右键粘贴、拖拽导入的主链收口
- 旧版本回退与兼容机制

3.2 本次明确不纳入范围

- AI 自动总结
- AI 自动拆节点
- AI 自动抽取标签
- AI 自动重写内容
- AI 内容质量评价
- 多人协同协议重构
- 导出协议重构

---

4. 当前版本实际现状

4.1 当前复制 / 粘贴主链路特点

当前版本的复制 / 粘贴 / 导入链路，核心更接近以下模式：

- 内部复制时，直接克隆 FreeFlow 原生画布元素
- 外部 HTML 粘贴时，做轻量清洗后直接落成文本元素
- 外部纯文本粘贴时，直接落成文本元素
- 文件拖拽时，按文件类型转成图片、文件卡片等元素

这说明当前系统已有可用能力，但“内部对象复制”和“外部内容导入”并不是同一套协议。

4.2 当前版本的主要优点

- 内部画布元素复制保真度高
- 普通文本、基础 HTML、文件拖拽已可用
- 系统已经具备文本元素、图片元素、文件卡片等原生落地能力
- 当前实现轻量，适合快速交付

4.3 当前版本的主要限制

- 没有统一的结构化中间协议
- 没有按来源拆分的专属解析器体系
- HTML 内容导入后仍偏“文本化”
- 表格、代码块、列表、多级结构保留能力有限
- Ctrl + V 与右键粘贴能力不完全一致
- 新增来源时容易继续在入口层堆分支

---

5. 当前版本与目标版本对比

对比项	当前版本	目标版本
输入入口	内部复制、原生粘贴、拖拽、右键粘贴各自分流	统一输入网关，再做来源识别与调度
解析方式	通用轻量清洗，按 MIME 粗分	来源专属解析器，按结构精细解析
中间层	原生 item 克隆 + HTML / 文本混用	全局唯一 canonical JSON
结构表达	主要是文本与少量样式保留	段落、列表、表格、代码块、图片、层级全部可表达
渲染方式	直接落成现有文本 / 图片 / 文件元素	从统一 JSON 生成原生块级与行内元素
扩展方式	新来源容易新增特判	新来源只需新增 parser
一致性	入口能力不一致	导入、复制、粘贴逐步统一
可维护性	中后期复杂度会持续上升	协议清晰，长期扩展成本更低

---

6. 新架构总览

新架构建议采用四层设计：

1. 输入层
2. 解析层
3. 统一中间层
4. 渲染层

统一主链建议收敛为：

`输入源 -> 来源识别 -> 对应解析器 -> canonical JSON -> 原生渲染器 -> FreeFlow 画布元素`

---

7. 输入层改造清单

7.1 目标

输入层负责承接所有外部与内部内容入口，把数据统一送入后续解析链路。

7.2 应覆盖的输入来源

- 画布内部复制
- 系统剪贴板粘贴
- 右键菜单粘贴
- 文件拖拽导入
- 文本拖拽导入
- HTML 富文本拖拽导入

7.3 本层要解决的问题

- 各入口能力不一致
- 某些入口只能拿纯文本，某些入口能拿 HTML / 文件
- 右键粘贴与 Ctrl + V 当前行为不统一
- 内部复制协议与外部粘贴协议割裂

7.4 输入层改造要求

- 新增统一 Import Gateway
- 为所有入口提供统一调用入口
- 每次导入都产生标准的输入描述对象
- 描述对象要带来源信息、MIME 信息、原始载荷和元数据

7.5 建议的输入描述对象

```
{
  entry: "paste" | "context-menu-paste" | "drag" | "internal-copy",
  sourceKind: "internal" | "html" | "plain-text" | "office" | "web" | "code" | "files",
  raw: {
    html: "",
    text: "",
    files: [],
    customData: null
  },
  meta: {
    timestamp: 0,
    fromApp: "",
    mimeTypes: []
  }
}
```

---

8. 解析层改造清单

8.1 目标

解析层负责把外部来源的原始数据拆开、清洗、抽取结构，并转换成统一协议可接收的结构内容。

8.2 解析层设计原则

- 各类来源解析器互不污染
- 解析逻辑只做“理解和归一”，不直接负责画布渲染
- 所有 parser 输出统一结构
- 无法完整保留时必须明确降级策略

8.3 建议纳入的解析器

8.3.1 通用 HTML 富文本解析器

目的：

- 处理一般网页复制、浏览器富文本、普通富文本编辑器内容
- 清除无效样式和垃圾标签
- 保留标题、段落、列表、链接、引用、图片、基础表格结构

8.3.2 网页正文解析器

目的：

- 面向网页文章与正文区域
- 清除广告、导航、脚注垃圾、无意义嵌套
- 保留正文层次、图文和表格

8.3.3 Office 富文本解析器

目的：

- 兼容 Word / WPS 等带私有样式的内容
- 尽可能保留多级标题、段落、字体样式、列表层级、表格
- 过滤 Office 专属冗余标记

8.3.4 代码文本解析器

目的：

- 严格保留源码字符、空格、缩进、空行、换行
- 识别 fenced code、缩进代码、纯代码片段
- 输出代码块结构，而不是普通段落文本

8.3.5 纯文本解析器

目的：

- 对无样式输入做最稳定处理
- 保留空行、换行、基本段落分隔

8.4 本层相较当前版本的核心提升

- 从“按 MIME 粗分”升级为“按来源与内容特征细分”
- 从“直接吃 HTML”升级为“先理解结构”
- 从“每次导入都临时处理”升级为“解析器可独立演进”

---

9. 统一中间层改造清单

9.1 目标

统一中间层是本次改造的核心。

所有解析器输出都必须落到这一层，后续所有渲染器都只认这一层，不直接认外部原始格式。

9.2 设计原则

- 全局唯一
- 来源无关
- 可版本化
- 可扩展
- 可序列化
- 可回退

9.3 建议的数据结构能力

统一协议至少应支持：

- 文档根节点
- 块级节点
- 行内节点
- 文本样式
- 段落与空行
- 标题层级
- 有序 / 无序列表
- 列表嵌套层级
- 表格与单元格
- 代码块与语言信息
- 图片节点
- 链接
- 引用块
- 分隔线
- 来源元数据
- schema version

9.4 建议的基础结构示意

```jsonc
{
  version: "1.0",
  source: {
    kind: "html",
    app: "browser"
  },
  document: {
    type: "doc",
    children: [
      {
        type: "heading",
        level: 1,
        children: [
          { type: "text", text: "标题", marks: [{ type: "bold" }] }
        ]
      },
      {
        type: "paragraph",
        children: [
          { type: "text", text: "正文内容" }
        ]
      },
      {
        type: "codeBlock",
        language: "js",
        text: "const a = 1;"
      }
    ]
  }
}
```

9.5 本层相较当前版本的意义

当前版本缺的是统一结构协议。

这一层补上后，系统才真正拥有以下能力：

- 不同来源共享同一渲染主链
- 新来源接入时不必重写渲染器
- 导入结果可调试、可比对、可测试
- 后续导出、再编辑、再复制都能统一处理

---

10. 渲染层改造清单

10.1 目标

渲染层负责把 canonical JSON 转成 FreeFlow 原生可编辑元素。

10.2 核心原则

- 不直接把外部 HTML 原样贴进画布当最终协议
- 尽量渲染成 FreeFlow 自己的原生元素
- 渲染后可拖拽、可编辑、可复制、可删除、可改样式

10.3 建议的渲染器模块

10.3.1 通用文本渲染器

负责：

- 段落
- 标题
- 普通行内样式
- 链接
- 引用块

10.3.2 列表渲染器

负责：

- 有序列表
- 无序列表
- 多级列表

10.3.3 表格渲染器

负责：

- 表格结构
- 单元格内容
- 行列组织

10.3.4 代码块渲染器

负责：

- 代码块容器
- 等宽排版
- 语言标识

10.3.5 图片渲染器

负责：

- 图片节点
- 尺寸与资源映射

10.4 本层相较当前版本的关键变化

- 当前更偏“直接生成文本元素”
- 目标版本更偏“按语义块生成对应原生元素”

这意味着最终画布上看到的不是一整坨被保留了部分样式的文本，而是一组可真正编辑和操作的原生结构块。

---

11. 新版本可新增的非 AI 特色能力

本节只写“结构化导入架构本身”可以带来的能力，不加入 AI 后处理。

11.1 双模式导入

同一份内容支持：

- 保真模式
- 结构模式

保真模式目标：

- 尽量还原原始排版与层级

结构模式目标：

- 按段落、列表、表格、代码块等原生块落地

11.2 来源识别与分流

系统可基于内容特征自动判断：

- 普通 HTML
- 网页正文
- Office 富文本
- 代码片段
- 纯文本

11.3 导入质量降级策略

当某类内容无法完整保留时，应能明确降级：

- 表格降级为分段文本
- 样式降级为基础粗斜体和字号
- 不支持的块级节点降级为段落

11.4 结构化复制基础

结构化导入落地后，后续内部复制也可逐步升级为：

- 原生元素复制
- canonical JSON 复制
- 文本 / HTML 兼容输出

11.5 可版本化协议

统一结构协议具备 schema version 后，可以支持：

- 老版本兼容读取
- 新版本平滑升级
- 测试样本长期复用

---

12. 与当前项目模块关系

12.1 当前可复用部分

- 现有内部复制链路
- 现有基础文本元素渲染能力
- 图片、文件卡片等现有元素类型
- 部分 HTML 清洗与字号规范化能力
- 现有拖拽和粘贴入口

12.2 当前需要新增的核心模块

- Import Gateway
- Source Detector
- Parser Registry
- Canonical Document Schema
- Import Diagnostics
- Structured Renderer Pipeline
- Fallback Strategy Manager

12.3 当前需要重构或收口的部分

- Ctrl + V 与右键粘贴入口差异
- 内部复制与外部导入的协议边界
- HTML 直接落文本元素的路径
- 代码片段被当普通文本处理的问题
- 表格和列表的扁平化问题

---

13. 分阶段实施建议

13.1 阶段 1：冻结旧链路，建立并行新架构

目标：

- 保留当前复制 / 粘贴 / 拖拽逻辑作为兼容层
- 新增 Import Gateway 和 parser 主框架
- 先不替换旧行为

阶段产出：

- 新入口网关
- 输入描述对象
- parser registry 空骨架
- canonical JSON 第一版协议

13.2 阶段 2：先接通收益最高的三类来源

建议优先：

- HTML 富文本
- 纯文本
- 代码文本

原因：

- 这三类覆盖面最大
- 风险相对可控
- 最容易建立中间层与渲染层闭环

13.3 阶段 3：接入网页正文与 Office 富文本

目标：

- 提升复杂内容保真度
- 收敛更多现实办公场景

13.4 阶段 4：逐步统一内部复制协议

目标：

- 保留原生 item 复制兼容能力
- 引入 canonical JSON 输出能力
- 让内部复制和外部导入开始共享更多主链

13.5 阶段 5：收口旧入口，形成正式主链

目标：

- 新架构成熟后再逐步把旧入口切到新链路
- 未覆盖内容继续走兼容降级

---

14. 兼容策略

14.1 旧链路必须保留

在新架构稳定前，当前版本的复制 / 粘贴 / 文件拖拽不可直接删除。

14.2 新入口必须支持回退

新 parser 解析失败时，应允许：

- 回退到旧 HTML 导入逻辑
- 回退到纯文本导入逻辑
- 回退到文件卡片导入逻辑

14.3 新协议必须带版本号

建议从第一版开始为 canonical JSON 增加 schema version，避免后续协议升级失控。

---

15. 测试与验收清单建议

建议后续测试样本至少覆盖：

- 浏览器正文复制
- 网页表格复制
- Word / WPS 富文本复制
- VSCode 代码复制
- 纯文本多段落复制
- 图片与文本混合拖拽
- 内部画布对象复制
- 右键粘贴与 Ctrl + V 行为对比

重点验收点：

- 结构保留是否稳定
- 字号与样式是否失真
- 列表层级是否正确
- 代码缩进是否完整
- 表格是否降级合理
- 渲染后是否仍可正常编辑

---

16. 风险与难点

16.1 主要技术难点

- Office 私有富文本兼容复杂
- 网页 HTML 垃圾标签和样式污染严重
- 代码块与普通段落的识别边界容易模糊
- 表格结构渲染需要新原生组件支持
- 当前项目已有复制链路较多，收口要谨慎

16.2 主要产品风险

- 新旧链路混用时行为可能不一致
- 某些极端内容可能短期内导入结果不如旧版本“看起来像”
- 结构模式与保真模式的选择策略需要明确

16.3 主要工程风险

- 协议设计过早固化会影响后续扩展
- 如果不先做兼容层隔离，容易把现有稳定链路带崩
- 如果 parser 和 renderer 职责不清，会重新回到入口堆逻辑的老问题

---

17. 预期收益

完成本次改造后，FreeFlow 相较当前版本将获得以下明显提升：

- 复制 / 粘贴 / 拖拽导入主链更统一
- 新来源接入成本显著下降
- 表格、列表、代码块、图文内容保留能力更强
- 外部内容能更稳定地转为 FreeFlow 原生结构
- 后续导出、内部复制、结构化编辑会有统一协议基础
- 系统整体从“能贴进来”升级为“能理解结构并落成自己的内容”

---

18. 最终结论

当前 FreeFlow 的复制 / 粘贴链路已经具备基础可用性，但整体仍属于“功能型实现”。

本文件定义的新方案，目标不是修补旧链路的几个具体问题，而是把整个系统升级为一套可扩展的结构化导入架构。

这一改造方向相较当前版本的本质差别在于：

- 当前版本重点在“把内容贴进来”
- 目标版本重点在“理解内容结构，再转成 FreeFlow 自己的原生表达”

因此，本次改造建议坚持以下路线：

- 先隔离旧版本
- 再并行建设新架构
- 先完成非 AI 的结构化导入主链
- 成熟后再考虑 AI 增强层接入

这条路线是当前项目在稳定性、扩展性、长期产品能力三者之间最均衡的方案。